EnterprateAI

Proposal Intelligence

Claude Code Implementation Pack

Development phases, implementation tickets, data model, APIs, event rules, UX acceptance criteria, security requirements and test catalogue

Document	Value
Version	1.0
Date	1 September 2026
Status	Development handoff baseline
Controlling product source	Proposal Intelligence PRD v1.1 and adopted end-to-end user journey
Primary implementation tool	Claude Code
Product owner	Enterprate Limited

IMPLEMENTATION BOUNDARY This pack translates the PRD into executable development work. It does not authorise Claude Code to redesign the architecture, replace existing modules, change commercial rules, expose private tenant data, or advance to a later phase without the applicable release gate.

1. How the Development Team Should Use This Pack

Place the Markdown version under the repository documentation folder, for example docs/proposal-intelligence/implementation-pack.md. Keep the Word version for product, design and management review. Do not replace an existing repository CLAUDE.md; merge only the approved operating rules after technical review.

1.1 Recommended Claude Code Workflow

1. Give Claude Code the PRD v1.1, this implementation pack, the adopted wireframes and the repository's existing CLAUDE.md or engineering instructions.
2. Start with PI-000 only. Require Claude Code to inspect and report the actual stack, architecture, tenancy, authentication, database, event, file-storage, contract and test patterns before writing feature code.
3. Approve the Phase 0 architecture decisions and migration plan. Do not allow implementation to begin while the technical baseline is unresolved.
4. Execute one ticket at a time on a dedicated branch or isolated change set. Claude Code must state the ticket ID in its plan and final report.
5. For each ticket, require tests, production build/type-check/lint checks, migration safety checks where applicable, and a concise diff summary.
6. Stop at every phase gate for product, security and engineering approval. Do not ask Claude Code to implement the entire pack in one prompt.

1.2 Repository Operating Rules for Claude Code

7. Inspect before changing: read repository instructions, package manifests, architecture docs, database schema, migrations, auth/tenant middleware, API conventions, UI component system, event infrastructure and tests.
8. Preserve the approved architecture and existing naming conventions. Propose an ADR when the repository lacks a required capability; do not silently introduce a parallel framework or service.
9. Use existing dependencies where suitable. Any new production dependency requires a reason, security/license review and explicit approval.
10. Never use destructive database commands, rewrite migration history, bypass tenant filters, weaken authentication, disable tests or use forceful dependency remediation.
11. Keep each ticket narrow. Do not refactor unrelated code, reformat broad areas, or implement future-phase functionality opportunistically.

12. All write endpoints enforce authorisation, validation, idempotency where relevant and an audit event. Client-side guards are never the security boundary.
13. All AI and document inputs are untrusted. Generated content remains a draft until an authorised user approves the action.
14. Run the repository's prescribed verification commands. If a check cannot run, stop and report the exact blocker rather than claiming completion.

1.3 Standard Claude Code Ticket Prompt

```
Implement ticket <TICKET_ID> only from the Proposal Intelligence Implementation Pack v1.0.

Before coding:
1. Read CLAUDE.md and all repository instructions.
2. Inspect the current implementation patterns relevant to this ticket.
3. State the files/components you expect to change, assumptions, migration impact and tests.
4. Stop and ask if the ticket conflicts with existing architecture or requires a new
dependency.

While coding:
- Preserve tenant isolation, RBAC, auditability and idempotency.
- Do not implement later tickets or refactor unrelated code.
- Add or update unit, integration and workflow tests specified by the ticket.

Before completion:
- Run formatting, lint, type-check, unit/integration tests, production build and migration
checks required by the repository.
- Run git diff --check and inspect the final diff for unrelated changes.
- Report acceptance criteria one by one as Passed, Failed or Not Run, with evidence.
- Stop after this ticket. Do not begin the next ticket.
```

2. Non-negotiable Product and Architecture Rules

Rule	Required implementation
Product position	EnterprateAI remains the Decision Intelligence and orchestration layer, not a full procurement, accounting, CRM or project-management product.
Module ownership	Business Blueprints owns document authoring; Business Operations owns live Sales, Procurement, Contracts, Transactions and Reports workflows; Marketplace owns discovery and Apply; Catalogue owns Offerings, Customers and Vendors.
RFP/RFQ placement	Canonical RFP/RFQ records live in Business Operations - Procurement. Any generated request document is indexed in Saved Documents and links back to the Procurement record.

Rule	Required implementation
Proposal preparation	Generated proposal drafts live in Business Blueprints - Proposals and Saved Documents. Submitted operational activity appears in proposer Sales and recipient Procurement against one canonical submission.
Paid access	Starter Insight or higher is required to generate or submit. Generate asks a non-user to create a free account before upgrade; Upload permits anonymous preparation but requires signup and upgrade at Submit.
Active proposal rule	One active ProposalSubmission per proposing business per ProposalRequest, excluding revisions. A revision creates a new immutable ProposalVersion under the same submission.
Human decisions	AI may analyse, rank and recommend but cannot automatically shortlist, reject, prefer, award or activate a contract.
Simulation boundary	Submitted, shortlisted and preferred proposals remain hypothetical. They must not mutate live financial or Fragility state.
Commercial conversion	Award links or creates existing Vendor and Customer records using Business IDs, links the Offering and creates a draft in the existing Contract feature.
Binding control	An authorised human must review and activate the Contract. Only confirmed contract and actual transaction events update live ontology and intelligence state.
Data privacy	A tenant sees only data deliberately shared in the proposal. Private financial, capacity, Fragility and internal recommendation data are never disclosed across tenants by default.
Source of truth	Operational status is owned by canonical Business Operations records. Saved Documents is an index and authoring repository, not a duplicate workflow state.

3. Delivery Phases and Release Gates

Phase	Scope	Mandatory exit gate
0 - Discovery and foundation	Repository discovery, architecture decisions, permissions, migration plan, event envelope, threat model, feature flags and observability.	Engineering, product, security and data owners approve the discovered architecture and migration/rollback plan.

Phase	Scope	Mandatory exit gate
1 - Workflow MVP	Settings, Procurement requests, Marketplace discovery, Upload, signup/upgrade-at-submit, submissions, versions, inboxes, statuses, communications, notifications and abuse controls.	Complete Upload journey passes tenant isolation, security, idempotency, accessibility and UAT tests.
2 - Paid generation and documents	Contextual upgrade, Catalogue Offering mapping, AI generation, source traceability, editor/preview and Saved Documents integration.	Entitlement, claim-traceability, retry/recovery and document-security tests pass.
3 - Evaluation and comparison	Criteria, mandatory gates, weighted evaluation, shortlist and explainable comparison.	Evaluator versioning, determinism and product acceptance thresholds are approved.
4 - Scenario intelligence	Proposal-to-scenario mapping, multi-scenario comparison, engine deltas and recommendations.	Automated proof confirms no live-state mutation; engine regression and explainability tests pass.
5 - Award and commercial conversion	Award, Vendor/Customer/Offering linkage, Opportunity/Demand Evidence, existing Contract draft and activation events.	Idempotency, partial-failure recovery, permissions and human approval tests pass.
6 - Pilot and release	Reports, analytics, moderation operations, controlled cohorts, performance and launch review.	Pilot thresholds, security sign-off, support readiness and go/no-go decision are documented.

4. Claude Code Implementation Ticket Register

Dependencies are ticket IDs. Each ticket is independently reviewable and must produce a narrow change set. The completion evidence column is the minimum; repository-specific checks discovered in PI-000 also apply.

4.1 Phase 0 - Discovery and Foundation

PI-000 - Repository architecture discovery

Dependencies: None

Scope: Produce docs/proposal-intelligence/technical-baseline.md covering framework/runtime, package manager, modules, database/ORM, migrations, auth, tenant resolution, RBAC, storage, queues/jobs, events, payments/entitlements, Contract/Catalogue integration, UI system, tests and deployment. No feature code.

Acceptance / evidence: Baseline cites actual files and commands; unknowns and conflicts are explicit; product rules are mapped to existing components.

PI-001 - Requirements traceability and ADR set

Dependencies: PI-000

Scope: Create a traceability matrix from PRD requirement IDs to planned modules, tickets, APIs and tests. Draft ADRs only for genuine architecture gaps.

Acceptance / evidence: Every Must requirement has an owner and test reference; no unapproved framework replacement.

PI-002 - Feature flags and configuration

Dependencies: PI-000

Scope: Add disabled-by-default flags for Marketplace exposure, request publishing, upload submission, generation, evaluation, intelligence and award orchestration.

Acceptance / evidence: Flags are server-enforced, tenant/cohort capable, auditable and tested for on/off behaviour.

PI-003 - Permissions and role policy

Dependencies: PI-000

Scope: Map existing roles to proposal.view, request.manage, proposal.submit, proposal.review, proposal.shortlist, proposal.negotiate, proposal.award, contract.activate and moderation permissions.

Acceptance / evidence: Default-deny policy is enforced server-side; unauthorised actions return the repository-standard 403 response and audit outcome.

PI-004 - Migration and rollback plan

Dependencies: PI-000, PI-001

Scope: Prepare forward-only schema migration sequence, backfill approach, indexes, rollback/disable strategy and production data checks.

Acceptance / evidence: Migrations run on an empty and representative database; rollback does not require destructive history rewrites.

PI-005 - Versioned event envelope and outbox

Dependencies: PI-000

Scope: Adopt or extend the existing event/outbox mechanism with version, tenant, actor, correlation and idempotency metadata.

Acceptance / evidence: Events commit atomically with state change or use the repository's proven equivalent; duplicate delivery is safe.

PI-006 - Threat model and privacy review

Dependencies: PI-000

Scope: Document trust boundaries for public Marketplace, anonymous upload, cross-tenant proposals, AI extraction/generation, downloads, payments and award orchestration.

Acceptance / evidence: Threats have controls, owners and tests; unresolved high risks block Phase 1.

PI-007 - Observability and analytics baseline

Dependencies: PI-000, PI-005

Scope: Define correlation IDs, structured logs, metrics, traces and privacy-safe analytics names before feature work.

Acceptance / evidence: No proposal body or sensitive extracted data is logged; dashboards/alerts can distinguish tenant-safe counts and failures.

4.2 Phase 1 - Workflow MVP

PI-101 - Proposal preferences data and service

Dependencies: PI-004

Scope: Implement current preference plus immutable audit versions for enabled state, accepted modes, categories, exclusions, eligibility, caps and visibility.

Acceptance / evidence: Multiple modes are supported; derived Marketplace status is deterministic; tenant scope and audit history tests pass.

PI-102 - Proposal settings screen

Dependencies: PI-003, PI-101

Scope: Build Business/Marketplace settings UI for Open for Proposals with mode multi-select and status preview.

Acceptance / evidence: Save validation, permission denial, loading, error and disabled states match the adopted UX; keyboard and label checks pass.

PI-103 - ProposalRequest, revision and requirement model

Dependencies: PI-004

Scope: Implement canonical request, immutable revisions and mandatory/weighted requirements. All submissions reference a request, including a system-owned evergreen context for unsolicited intake.

Acceptance / evidence: General, specific and unsolicited contexts are represented without null-rule ambiguity; revision history is immutable.

PI-104 - Procurement request builder

Dependencies: PI-003, PI-103

Scope: Build Business Operations - Procurement create/edit flow for General and Specific requests, including budget visibility, criteria, deadline, cap and invite-only controls.

Acceptance / evidence: Required fields vary correctly by type; drafts are private; no Marketplace publication occurs before explicit Publish.

PI-105 - Request publish, revise, pause and close service

Dependencies: PI-103, PI-104, PI-005

Scope: Implement server-side lifecycle and events for Draft, Published, Paused, Closed and Expired plus request revisions.

Acceptance / evidence: Invalid transitions fail; active proposers receive the defined revision notification; closed/expired requests reject new submissions.

PI-106 - Marketplace Open for Proposals badge and filters

Dependencies: PI-101, PI-105

Scope: Show the derived status on eligible business cards/profiles and expose proposal-type/category filters.

Acceptance / evidence: Disabled businesses are not shown as open; public output contains no private preference or tenant data.

PI-107 - Marketplace request detail and Apply entry

Dependencies: PI-105, PI-106

Scope: Build public request details and Apply action for published general/specific requests plus eligible unsolicited intake.

Acceptance / evidence: Visibility, invitation, deadline and cap rules are enforced server-side; inaccessible requests return safe responses.

PI-108 - Generate-or-Upload method chooser

Dependencies: PI-107

Scope: Implement the modal/page presenting Generate with EnterprateAI and Upload Proposal with paid-plan disclosure.

Acceptance / evidence: Both routes preserve business/request context; disclosure is visible before selection; focus management and Escape behaviour pass.

PI-109 - Anonymous upload session

Dependencies: PI-108, PI-006

Scope: Create short-lived anonymous upload sessions with opaque token, request binding, checksum, expiry and no tenant membership.

Acceptance / evidence: Token cannot enumerate or access other uploads; expiry and deletion are tested; file is not downloadable before scan clearance.

PI-110 - File validation, quarantine and malware scan

Dependencies: PI-109

Scope: Allow PDF and non-macro DOCX within approved size; quarantine first; scan asynchronously; record scan status.

Acceptance / evidence: Blocked types, macro files, oversize, malware and scan failures never reach extraction or recipient access.

PI-111 - Upload extraction and correction preview

Dependencies: PI-110

Scope: Extract the minimum structured proposal schema in isolation and show fields, missing data and confidence for user correction.

Acceptance / evidence: Original remains preserved; prompt-injection text is treated as data; corrections are saved without altering the source file.

PI-112 - Signup binding for anonymous upload

Dependencies: PI-109, PI-111

Scope: At Submit, create/login account then atomically bind the upload and request to the verified business/user.

Acceptance / evidence: A token is single-claim; interrupted signup preserves progress; cross-account claim attempts fail and are logged.

PI-113 - Subscription entitlement at submission

Dependencies: PI-112

Scope: Check Starter Insight-or-higher entitlement only when Upload user submits; preserve state through payment and return to final confirmation.

Acceptance / evidence: Free users cannot submit; paid users continue; payment cancel/failure preserves draft; return state cannot be tampered with.

PI-114 - ProposalSubmission and immutable ProposalVersion

Dependencies: PI-103, PI-004

Scope: Implement canonical submission and version snapshots, attachments and checksum. Enforce one active submission per proposer/request.

Acceptance / evidence: Concurrent duplicate creation is prevented by database constraint; revisions increment version under the same submission.

PI-115 - Submission validation pipeline

Dependencies: PI-110, PI-111, PI-113, PI-114

Scope: Validate identity, entitlement, profile, relevance, fields, scan, duplicate, deadline, cap and active-submission rule in a deterministic pipeline.

Acceptance / evidence: All failures use stable codes and user-safe messages; validation order cannot leak private recipient information.

PI-116 - Idempotent submit transaction

Dependencies: PI-114, PI-115, PI-005

Scope: Freeze the confirmed version, set Submitted, create audit/outbox event and return the canonical result under an idempotency key.

Acceptance / evidence: Retries return the same submission/version; partial writes do not occur; concurrent submits have one outcome.

PI-117 - Recipient Procurement proposal inbox

Dependencies: PI-116, PI-003

Scope: Build tenant-scoped inbox with filters, unread state, status, deadline and safe summary.

Acceptance / evidence: Only recipient members with review permission can access; pagination/filtering are server-side and stable.

PI-118 - Proposer Sales activity view

Dependencies: PI-116, PI-003

Scope: Show the proposer its submitted proposal activity under Business Operations - Sales using the same canonical submission.

Acceptance / evidence: No duplicate operational status is stored; deep links open exact version/request; other proposer users require permission.

PI-119 - Proposal detail, source and event timeline

Dependencies: PI-117, PI-118

Scope: Build role-aware detail with structured data, cleared attachment, versions and material event history.

Acceptance / evidence: Recipient cannot see proposer-private analysis; proposer cannot see recipient-only evaluation; access is audited.

PI-120 - Clarification and revision communication

Dependencies: PI-119, PI-005

Scope: Implement typed messages for clarification, response and revision request against proposal/version.

Acceptance / evidence: Messages are immutable or correction-audited; notification and permission tests pass; attachments use the same scan controls.

PI-121 - Canonical status transition service

Dependencies: PI-114, PI-120

Scope: Enforce the approved state machine server-side with actor, reason, prior/next state and version in every event.

Acceptance / evidence: Arbitrary client status writes are impossible; terminal and role-invalid transitions fail consistently.

PI-122 - Revision, withdrawal, decline and archive flows

Dependencies: PI-121

Scope: Create new version for revisions; implement proposer withdrawal and recipient decline/archive with reason rules.

Acceptance / evidence: Revisions do not consume another submission; frozen versions remain unchanged; audit history remains accessible.

PI-123 - In-app and email notifications

Dependencies: PI-005, PI-120, PI-121

Scope: Implement event-driven notifications for submit, clarification, revision, status, deadline and closure using existing preferences.

Acceptance / evidence: Duplicate events do not send duplicate notifications; no confidential proposal body appears in email.

PI-124 - Report, block and moderation intake

Dependencies: PI-003, PI-119

Scope: Allow recipient to report proposal/proposer and block future intake under an auditable moderation workflow.

Acceptance / evidence: Block applies prospectively without deleting history; moderator access is least privilege; report abuse is rate-limited.

PI-125 - Phase 1 end-to-end hardening

Dependencies: PI-101 to PI-124

Scope: Complete accessibility, tenant, concurrency, recovery, performance and full Upload workflow tests; document operational runbook.

Acceptance / evidence: Phase 1 exit suite passes and product/security UAT evidence is attached; no Phase 2 code is included.

4.3 Phase 2 - Paid Generation and Saved Documents

PI-201 - Contextual signup and upgrade for Generate

Dependencies: PI-108, PI-002

Scope: Require signup before Generate, save opportunity, then show proposal-specific Starter Insight upgrade before generation.

Acceptance / evidence: No proposal generation occurs on Free; payment return restores exact request; generic pricing redirect is not used.

PI-202 - Catalogue Offering selection and creation link

Dependencies: PI-201

Scope: Select an eligible tenant-owned Offering or enter the existing Catalogue creation flow and return safely.

Acceptance / evidence: Only authorised Offerings are accessible; unpublished/private Offering data is used only with proposer approval.

PI-203 - Approved-source context and missing-input mapper

Dependencies: PI-202

Scope: Map request requirements to approved business/Offering data and produce missing/confirmation questions with provenance.

Acceptance / evidence: Every populated claim has source reference or explicit user input; private data is not inserted silently.

PI-204 - Asynchronous proposal generation job

Dependencies: PI-203

Scope: Queue generation with versioned prompt/model/config, progress, cancellation/retry and cost/credit handling.

Acceptance / evidence: Web request is not held open; retries are idempotent; failed generation never creates a submitted version.

PI-205 - Proposal editor, preview and approval

Dependencies: PI-204

Scope: Build Business Blueprints - Proposals editor with structured sections, preview, save and explicit user approval.

Acceptance / evidence: Draft remains editable; approval freezes the candidate version; unsupported fields show warnings.

PI-206 - Claim provenance and hallucination controls

Dependencies: PI-203, PI-205

Scope: Persist field-level provenance and confirmation state; block unconfirmed material claims at submit according to policy.

Acceptance / evidence: Generated claims are traceable; model instructions cannot override product/security rules; tests cover fabricated evidence.

PI-207 - Saved Documents index and deep links

Dependencies: PI-205, PI-114

Scope: Create/update DocumentRecord for proposal drafts and submitted versions with operational usage links.

Acceptance / evidence: Saved Documents never owns operational status; archive does not delete active Sales/Procurement/Contract records.

PI-208 - Extraction quality and resumable recovery

Dependencies: PI-111, PI-207

Scope: Add confidence thresholds, field-level errors, re-extraction and safe resume for upload jobs.

Acceptance / evidence: Re-extraction creates audit/version metadata; user corrections are never silently overwritten.

PI-209 - Phase 2 entitlement and document hardening

Dependencies: PI-201 to PI-208

Scope: Run paid-access, credits, payment return, source provenance, storage, AI safety and recovery suites.

Acceptance / evidence: Free-plan bypass, duplicate credit charge, cross-tenant source access and prompt injection tests all fail safely.

4.4 Phase 3 - Evaluation and Comparison

PI-301 - Evaluation criteria and ruleset versioning

Dependencies: PI-103

Scope: Implement mandatory and weighted criteria, scale, guidance, default templates and immutable ruleset versions.

Acceptance / evidence: Weights validate deterministically; active evaluations retain the exact ruleset version.

PI-302 - Mandatory compliance gate

Dependencies: PI-301, PI-114

Scope: Evaluate pass/fail requirements against the exact proposal version and source evidence.

Acceptance / evidence: Failure is visible and cannot be hidden by aggregate score; uncertain evidence is Not Determined, not Pass.

PI-303 - Weighted evaluation service

Dependencies: PI-301, PI-302

Scope: Score fit, commercial value, feasibility, evidence, operational risk and strategic impact with confidence.

Acceptance / evidence: Same inputs/ruleset produce the same deterministic score; rounding and missing-data rules are tested.

PI-304 - Shortlist action and permission

Dependencies: PI-121, PI-303

Scope: Allow authorised recipient to shortlist eligible versions with optional override reason for flagged proposals.

Acceptance / evidence: AI never shortlists; exact version and actor are recorded; invalid/terminal proposals cannot be shortlisted.

PI-305 - Shortlist comparison screen

Dependencies: PI-303, PI-304

Scope: Compare selected versions by requirements, commercial terms, evidence, missing data, scores and confidence.

Acceptance / evidence: No winner is implied when data is insufficient; currency/units are explicit; keyboard and responsive tests pass.

PI-306 - Explainable evaluation narrative

Dependencies: PI-303

Scope: Generate a source-grounded explanation of scores, trade-offs, assumptions and missing information.

Acceptance / evidence: Narrative cannot change deterministic scores; every statement links to evaluation output/source; recipient-only privacy holds.

PI-307 - Phase 3 evaluator regression suite

Dependencies: PI-301 to PI-306

Scope: Create fixed fixtures for edge cases and approved evaluator expectations.

Acceptance / evidence: Ruleset changes require explicit fixture updates and review; no nondeterministic score drift.

4.5 Phase 4 - Scenario and Decision Intelligence

PI-401 - Proposal-to-scenario mapping

Dependencies: PI-303

Scope: Map price, terms, costs, benefits, resources, staffing, dependency and timing into versioned hypothetical ScenarioChanges.

Acceptance / evidence: Mapping preserves currency/time horizon and flags unsupported assumptions; no live records are updated.

PI-402 - Existing simulation-engine adapter

Dependencies: PI-000, PI-401

Scope: Integrate through the existing stateless/non-mutating simulation contract; do not create a second engine.

Acceptance / evidence: Adapter input/output is versioned and tenant-scoped; failures are recoverable without proposal mutation.

PI-403 - Base/downside/expected/upside runs

Dependencies: PI-402

Scope: Generate and execute four labelled scenarios using explicit assumptions and confidence.

Acceptance / evidence: All runs reference exact proposal/evaluation versions; missing assumptions are visible; results are repeatable.

PI-404 - Engine impact deltas

Dependencies: PI-403

Scope: Present Viability, Survival, Stability, Growth and Fragility deltas only where supported by existing engines.

Acceptance / evidence: Unavailable metrics are Not Available; hypothetical deltas are visually distinct from live scores.

PI-405 - Decision recommendation service

Dependencies: PI-303, PI-403, PI-404

Scope: Rank or state no recommendation with reasons, trade-offs, assumptions, missing data, confidence and risk deltas.

Acceptance / evidence: Service cannot call shortlist/prefer/award endpoints; low confidence and mandatory failures are prominent.

PI-406 - Proposer-side deliverability intelligence

Dependencies: PI-203, PI-402

Scope: Privately assess profitability, cash-before-payment, staffing/capacity, timing and concentration risk before submission.

Acceptance / evidence: Output is proposer-only unless deliberately shared; live proposer state is not mutated.

PI-407 - No-live-mutation proof

Dependencies: PI-401 to PI-406

Scope: Add database/state snapshots and contract tests proving proposal evaluation/simulation never alters live operational or financial records.

Acceptance / evidence: Automated suite detects any write outside approved scenario/assessment tables and blocks release.

PI-408 - Phase 4 security, regression and observability

Dependencies: PI-401 to PI-407

Scope: Harden intelligence jobs, provenance, access, performance, cost and failure monitoring.

Acceptance / evidence: Engine/model/ruleset versions and correlation IDs are retained without logging confidential payloads.

4.6 Phase 5 - Award and Commercial Conversion

PI-501 - Award confirmation UI and permission

Dependencies: PI-304, PI-405

Scope: Require authorised user to confirm the exact version, rationale and non-binding-to-contract warning.

Acceptance / evidence: AI cannot invoke Award; stale version confirmation fails; double-submit is safe.

PI-502 - ProposalDecision and awarded-version freeze

Dependencies: PI-501, PI-005

Scope: Persist Award decision against exact immutable version and emit proposal.awarded.

Acceptance / evidence: Only allowed request policy can award; decision is auditable; later changes require controlled revision/contract amendment.

PI-503 - Vendor link-or-create

Dependencies: PI-502

Scope: Find by unique Business ID within requesting tenant Catalogue; link existing Vendor or create from approved Marketplace data.

Acceptance / evidence: Name-only matching is prohibited; retries do not duplicate; minimal approved data only.

PI-504 - Customer link-or-create

Dependencies: PI-502

Scope: Find/link/create requesting business as Customer in proposer Catalogue using Business ID.

Acceptance / evidence: Same idempotency, tenancy and approved-data rules as Vendor; cross-tenant writes are service-authorised and audited.

PI-505 - Opportunity and Demand Evidence linkage

Dependencies: PI-502

Scope: Create/update proposer Opportunity and verified evidence signals according to existing ontology rules.

Acceptance / evidence: Submission/shortlist/preferred/award signals retain strength and source; recipient request is not misclassified as proposer Opportunity.

PI-506 - Existing Contract draft mapping

Dependencies: PI-503, PI-504

Scope: Open existing Business Operations - Contracts workflow and prepopulate parties, scope, deliverables, price, terms, dates, assumptions, exclusions and references.

Acceptance / evidence: A draft only is created; no signature/activation occurs automatically; exact awarded version is linked.

PI-507 - Idempotent award orchestrator

Dependencies: PI-502 to PI-506

Scope: Coordinate link/create and Contract draft steps with durable status, idempotency and correlation ID.

Acceptance / evidence: Retries converge on one Vendor, Customer, Opportunity and Contract; partial success is visible and recoverable.

PI-508 - Integration failure recovery

Dependencies: PI-507

Scope: Provide authorised retry/compensation controls and support diagnostics without exposing private data.

Acceptance / evidence: Award remains recorded while integration is pending; no duplicate or silent rollback; recoverable errors are classified.

PI-509 - Contract activation and transaction events

Dependencies: PI-506

Scope: On existing Contract activation, set proposal Contracted and update confirmed commitments; only actual transaction events update actual financial state.

Acceptance / evidence: Proposal award alone does not mutate live state; activation is authorised and idempotent; transaction linkage follows existing validation.

PI-510 - Award/contract notifications and audit

Dependencies: PI-507, PI-509

Scope: Notify both parties and record orchestration/activation outcomes using privacy-safe templates.

Acceptance / evidence: Duplicate events do not duplicate notifications; audit links exact decision, version and Contract.

PI-511 - Phase 5 end-to-end conversion suite

Dependencies: PI-501 to PI-510

Scope: Test Award through Contracted plus every partial-failure and permission path.

Acceptance / evidence: Commercial conversion gate passes with one canonical record set and mandatory human contract approval.

4.7 Phase 6 - Pilot and Release

PI-601 - Operations Reports dashboards

Dependencies: PI-125, PI-209, PI-307, PI-408, PI-511

Scope: Add Business Operations - Reports measures for pipeline, response, shortlist, award, contract and quality, using tenant-safe aggregates.

Acceptance / evidence: Metrics reconcile with canonical records; filters and export follow existing permissions.

PI-602 - Product analytics and privacy review

Dependencies: PI-007, PI-601

Scope: Instrument funnel events from discovery through contract while excluding confidential bodies and private intelligence.

Acceptance / evidence: Consent/policy requirements are met; analytics events are versioned and deduplicated.

PI-603 - Moderation operations and SLA

Dependencies: PI-124

Scope: Provide moderator queue, evidence-safe view, actions, appeal and audit/runbook.

Acceptance / evidence: Moderator tenancy exception is explicit, least-privilege and audited; no bulk content exposure.

PI-604 - Pilot cohorts and plan allowances

Dependencies: PI-002

Scope: Configure verified pilot tenants, plan allowances, intake caps and staged feature activation.

Acceptance / evidence: Non-pilot tenants remain unaffected; revisions never consume new-request allowance.

PI-605 - Load, reliability and recovery testing

Dependencies: PI-601 to PI-604

Scope: Exercise Marketplace, upload, jobs, inbox, events and orchestration under expected and burst load.

Acceptance / evidence: Budgets discovered in PI-000 are met or deviations are approved; queues recover without duplicate outcomes.

PI-606 - Release evidence and go/no-go review

Dependencies: All prior gates

Scope: Compile security, privacy, accessibility, performance, migration, rollback, UAT, pilot metrics and unresolved risks.

Acceptance / evidence: Named owners approve release; disabled-by-default rollback remains available; open high risks block launch.

5. Logical Database and Ontology Changes

IMPLEMENTATION INSTRUCTION The following is a logical contract. Claude Code must map it to the repository's existing database and ORM conventions discovered in PI-000. Table names may be adapted consistently, but entity meaning, ownership, immutability, tenant constraints and uniqueness rules must be preserved.

Logical entity	Purpose	Essential fields	Constraints / ownership
proposal_preferences	Current business preference	id, tenant_id, business_id, enabled, accepted_modes[], categories[], exclusions[], eligibility_json, receipt_cap, visibility, version, updated_by/at	Unique business_id; current projection only; every update writes preference version.
proposal_preference_versions	Immutable preference audit	id, preference_id, version_no, snapshot_json, changed_by/at, correlation_id	Unique preference_id + version_no; append-only.
proposal_requests	Canonical unsolicited/general/specific request	id, tenant_id, requester_business_id, request_type, title, status, current_revision_id, budget_visibility, currency, deadline, receipt_cap, visibility, award_policy, created_by/at	All submissions require request_id. One system evergreen unsolicited request per recipient/scope as needed.
proposal_request_revisions	Immutable published/draft request snapshot	id, request_id, revision_no, need, scope, deliverables_json, budget_json, dates_json, eligibility_json, criteria_version, created_by/at, published_at	Unique request_id + revision_no; submitted proposals retain the revision they answered.
proposal_requirements	Mandatory/scored requirement	id, request_revision_id, code, description, requirement_type, mandatory, weight, evidence_required, sort_order	Weights validated per ruleset; immutable after revision publish.

Logical entity	Purpose	Essential fields	Constraints / ownership
proposal_submissions	One active proposal relationship	id, recipient_tenant_id, proposer_tenant_id, request_id, request_revision_id, proposer_business_id, recipient_business_id, offering_id, current_version_id, status, submitted_at, terminal_at	Partial/composite uniqueness enforces one active submission per proposer_business_id + request_id; revisions are not new rows.
proposal_versions	Immutable submitted/revised snapshot	id, submission_id, version_no, method, structured_payload_json, source_document_id, checksum, extraction_confidence, approval_state, created_by/at, submitted_by/at	Unique submission_id + version_no; frozen after submission; exact JSON schema version stored.
proposal_attachments	Source/supporting file metadata	id, version_id/upload_session_id, storage_key, original_name, mime_type, size_bytes, checksum, scan_status, scan_provider_ref, retention_until, created_at	Private storage; no cleared download until scan_status=clean.
anonymous_upload_sessions	Pre-account upload state	id, opaque_token_hash, request_id, attachment_id, state, extracted_draft_json, expires_at, claimed_by_user/business, claimed_at	Single claim; short TTL; token stored hashed; no tenant membership until claim.
proposal_messages	Clarification/revision/negotiation thread	id, submission_id, version_id, sender_tenant/user, message_type, body, visibility_scope, created_at	Append-only or correction event; cross-tenant access only to proposal parties.
evaluation_criteria_sets	Versioned scoring rules	id, request_revision_id, version_no, dimensions_json, weight_total, created_by/at	Immutable when used; deterministic ruleset version.
proposal_evaluations	Evaluation result	id, proposal_version_id, criteria_set_id, ruleset_version, mandatory_results_json, scores_json, confidence, missing_data_json, status, completed_at	Unique active result per version/ruleset; rerun creates new version/result.

Logical entity	Purpose	Essential fields	Constraints / ownership
proposal_decisions	Preferred/decline/award decision	id, submission_id, proposal_version_id, decision_type, rationale, actor_user, actor_tenant, created_at	Award references exact frozen version; append-only decision history.
proposal_events	Immutable audit/event projection	id, tenant_scope, submission/request/document_id, event_type, event_version, actor_type/id, prior_status, next_status, payload_metadata_json, correlation_id, idempotency_key, occurred_at	No confidential body; append-only; unique producer + idempotency key where applicable.
document_records	Saved Documents index	id, tenant_id, document_type, current_document_version_id, authoring_status, operational_entity_type/id, owner_user, updated_at, archived_at	Does not own operational status; deep link to canonical record.
proposal_orchestration_runs	Award conversion state	id, proposal_decision_id, state, vendor_id, customer_id, opportunity_id, contract_id, step_results_json, correlation_id, idempotency_key, retry_count, created/updated_at	Unique awarded decision; retry-safe; partial state observable.

5.1 Required Enumerations

Enumeration	Values / rule
proposal_request_type	UNSOLICITED_EVERGREEN, SOLICITED_GENERAL, SOLICITED_SPECIFIC
request_status	DRAFT, PUBLISHED, PAUSED, CLOSED, EXPIRED, ARCHIVED
proposal_method	GENERATED, UPLOADED
proposal_status	DRAFT, SUBMITTED, VIEWED, UNDER_REVIEW, CLARIFICATION_REQUESTED, REVISION_REQUESTED, REVISED, SHORTLISTED, PREFERRED, NEGOTIATION, AWARDED, CONTRACT_DRAFTED, CONTRACTED, DECLINED, WITHDRAWN, EXPIRED, ARCHIVED
scan_status	PENDING, CLEAN, BLOCKED, FAILED, QUARANTINED
budget_visibility	HIDDEN, RANGE, EXACT

Enumeration	Values / rule
document_type	BUSINESS_PLAN, PROPOSAL, SALES_DOCUMENT, REQUEST_DOCUMENT
operational_entity_type	NONE, SALES_ACTIVITY, PROCUREMENT_REQUEST, PROPOSAL_SUBMISSION, CONTRACT, LIVE_BUSINESS_PLAN
decision_type	PREFERRED, DECLINED, AWARDED
orchestration_state	PENDING, RUNNING, PARTIAL_FAILURE, RETRYABLE, COMPLETED, TERMINAL_FAILURE

5.2 Critical Database Constraints

15. Every tenant-owned table includes tenant scope compatible with the repository's existing row-isolation pattern; tenant ID is never accepted blindly from the browser.
16. One active submission per proposer_business_id and request_id is enforced in the database, not only application code. Terminal statuses are Declined, Withdrawn, Expired and Archived; Contracted is historical but remains the same canonical submission.
17. A revision creates proposal_versions.version_no + 1 and updates current_version_id transactionally; the earlier version is immutable.
18. All recipient/proposer cross-tenant reads require both party relationship and user permission. Composite foreign keys or equivalent service checks prevent ID swapping.
19. Award and orchestration use unique decision/idempotency constraints to prevent duplicate Vendor, Customer, Opportunity and Contract records.
20. Soft deletion/archival follows existing policy. Submitted versions, decisions, audit events and contract references cannot be hard-deleted through normal user actions.
21. File checksum, MIME detection and scan status are server-produced. Original filename and browser-supplied content type are metadata only.

5.3 Ontology Relationships

Relationship	Rule
Business -> ProposalPreference	One current preference and many immutable preference versions.
Business -> ProposalRequest	Requesting business owns many requests; unsolicited intake is represented by a canonical evergreen request context.

Relationship	Rule
ProposalRequest -> ProposalSubmission	One-to-many; each proposer may hold only one active submission for that request.
ProposalSubmission -> ProposalVersion	One-to-many immutable versions; current_version_id identifies the active snapshot.
ProposalVersion -> Offering	References the proposer's existing/new Catalogue Offering where applicable.
ProposalVersion -> Assessment/Scenario	Evaluation and simulation always bind to exact version and ruleset/model version.
ProposalDecision -> Contract	Award decision creates one existing Contract draft through orchestration; human activation remains separate.
Business pair -> Vendor/Customer	At award, proposer is Vendor to requester and requester is Customer to proposer; match by Business ID.
Proposal -> Opportunity/DemandEvidence	Proposer-side Opportunity and progressively stronger evidence signals are linked with verification state.
DocumentRecord -> operational entity	DocumentRecord indexes authored content and deep-links to the canonical operational record without duplicating its status.

6. API Contract Baseline

API CONVENTION Paths below are logical REST contracts. Claude Code must adapt them to the repository's established route/versioning conventions while preserving behaviour, permissions, validation codes and idempotency. Never expose internal tenant IDs or storage keys to public clients.

Method	Logical route	Authorisation	Contract
GET	/proposal-preferences/me	Authenticated business member	Read current settings and derived Marketplace status.
PUT	/proposal-preferences/me	proposal.request.manage	Validate/update settings; emit preference.updated.

Method	Logical route	Authorisation	Contract
POST	/procurement/requests	proposal.request.manage	Create draft General/Specific request.
GET	/procurement/requests	Recipient tenant	List tenant requests with filters/pagination.
GET	/procurement/requests/{requestId}	Owner or permitted public projection	Return role-safe request/revision.
PATCH	/procurement/requests/{requestId}	proposal.request.manage	Update draft only; optimistic concurrency.
POST	/procurement/requests/{requestId}/publish	proposal.request.manage	Freeze revision and publish idempotently.
POST	/procurement/requests/{requestId}/revisions	proposal.request.manage	Create controlled revision and notifications.
POST	/procurement/requests/{requestId}/pause|close	proposal.request.manage	Server-side lifecycle transition.
GET	/marketplace/proposal-opportunities	Public	List safe published projections and filters.
GET	/marketplace/proposal-opportunities/{publicId}	Public/invite-aware	Read safe opportunity details.
POST	/proposal-upload-sessions	Public rate-limited	Create opaque request-bound anonymous session.
POST	/proposal-upload-sessions/{token}/attachments	Session token	Upload to quarantine using approved transfer method.
GET	/proposal-upload-sessions/{token}	Session token	Read scan/extraction/progress-safe state.
PATCH	/proposal-upload-sessions/{token}/draft	Session token	Save corrected extracted fields.
POST	/proposal-upload-sessions/{token}/claim	Authenticated verified user	Atomically bind single-use session to business.

Method	Logical route	Authorisation	Contract
POST	/proposal-submissions	Eligible paid proposer	Create canonical draft/submission context; idempotency required.
GET	/proposal-submissions/{id}	Proposal party with permission	Return role-filtered details and current version.
POST	/proposal-submissions/{id}/versions	Proposer submit permission	Create revision candidate/frozen version.
POST	/proposal-submissions/{id}/submit	Eligible paid proposer	Validate and idempotently freeze/deliver version.
POST	/proposal-submissions/{id}/actions/{action}	Role/action permission	Viewed, review, clarification, revision, shortlist, preferred, negotiate, decline, withdraw, archive via transition service.
POST	/proposal-submissions/{id}/messages	Proposal party	Add typed, sanitised, rate-limited message.
GET	/procurement/proposals	proposal.review	Recipient inbox; tenant-scoped filters and pagination.
GET	/sales/proposals	proposal.submit/view	Proposer activity using canonical status.
POST	/proposal-generation/jobs	Eligible paid proposer	Start generation for request + Offering; idempotency/credits.
GET	/proposal-generation/jobs/{id}	Job owner	Progress/result/error; no cross-tenant access.
POST	/proposal-evaluations	proposal.review	Evaluate exact version against criteria ruleset.
POST	/proposal-comparisons	proposal.review	Compare authorised shortlisted versions.
POST	/proposal-scenarios	proposal.review	Create non-mutating scenario set for exact versions.
POST	/proposal-recommendations	proposal.review	Return explainable recommendation or no-recommendation.

Method	Logical route	Authorisation	Contract
POST	/proposal-submissions/{id}/award	proposal.award	Confirm exact version/rationale; idempotency required.
GET	/proposal-orchestrations/{decisionId}	Authorised proposal party/support	Return safe award-conversion status.
POST	/proposal-orchestrations/{decisionId}/retry	Authorised support/owner	Retry only failed idempotent steps.

6.1 Standard Response and Error Envelopes

```
Success
{
  "data": { ... },
  "meta": {
    "requestId": "req_...",
    "correlationId": "corr_...",
    "version": 1
  }
}

Error
{
  "error": {
    "code": "PROPOSAL_PLAN_INELIGIBLE",
    "message": "Starter Insight or a higher plan is required to submit.",
    "fieldErrors": [],
    "retryable": false
  },
  "meta": {
    "requestId": "req_...",
    "correlationId": "corr_..."
  }
}
```

6.2 Stable Domain Error Codes

Code	Meaning / HTTP treatment
PROPOSAL_REQUEST_NOT_OPEN	Request paused, closed or expired; 409 or repository-standard domain conflict.
PROPOSAL_REQUEST_CAP_REACHED	Recipient intake cap reached; 409 without revealing private counts.
PROPOSAL_ALREADY_ACTIVE	Proposer already has an active submission for request; return canonical submission link where authorised.

Code	Meaning / HTTP treatment
PROPOSAL_PLAN_INELIGIBLE	Paid entitlement missing; 403/402 according to existing payment convention.
PROPOSAL_PROFILE_INCOMPLETE	Verified business/profile requirement not met; 422 with safe missing fields.
PROPOSAL_FILE_UNSAFE	Blocked/quarantined file; 422 with no malware internals.
PROPOSAL_VALIDATION_FAILED	Structured fields/mandatory submission requirements fail; 422.
PROPOSAL_VERSION_STALE	Client acts on non-current version; 409 and return current safe version metadata.
PROPOSAL_TRANSITION_INVALID	Status/action not permitted from current state; 409.
PROPOSAL_PERMISSION_DENIED	User lacks role/action permission; 403.
PROPOSAL_NOT_FOUND	Safe 404 for inaccessible or absent cross-tenant resources.
PROPOSAL_JOB_PENDING	Asynchronous job incomplete; 202/status projection.
PROPOSAL_ORCHESTRATION_PARTIAL	Award recorded but commercial conversion requires retry; 202/409 according to existing job pattern.

6.3 Idempotency and Concurrency

22. Require Idempotency-Key or the repository equivalent for upload claim, submit, publish, generation start, award and orchestration retry.
23. Scope keys to authenticated principal/anonymous session, operation and resource; store request fingerprint and canonical response.
24. Reject reuse of the same key with a different request body.
25. Use optimistic concurrency/version fields for editable preferences, requests and drafts; return stable stale-version conflict.
26. Database uniqueness remains the final guard against concurrent duplicate submissions and commercial records.

7. Event Contracts and Status Rules

7.1 Versioned Event Envelope

```
{
  "eventId": "evt_...",
  "eventType": "proposal.submitted",
  "eventVersion": 1,
  "occurredAt": "2026-09-01T10:00:00Z",
  "producer": "proposal-service",
  "tenantScope": ["recipient-tenant-id", "proposer-tenant-id"],
  "actor": {"type": "USER", "id": "user-id", "tenantId": "tenant-id"},
  "subject": {
    "requestId": "prq_...",
    "submissionId": "ps_...",
    "proposalVersionId": "pv_..."
  },
  "correlationId": "corr_...",
  "causationId": "evt_or_request_...",
  "idempotencyKey": "...",
  "data": {"priorStatus": "DRAFT", "nextStatus": "SUBMITTED"}
}
```

7.2 Required Events

Domain	Events	Primary consumers
Preference/request	preference.updated; request.draft_created; request.published; request.revised; request.paused; request.closed; request.expired	Marketplace projection, notifications, analytics, Saved Documents
Document	document.saved; document.version_created; document.archived	Saved Documents index, authoring UI, audit
Proposal	proposal.draft_created; proposal.uploaded; proposal.generated; proposal.submitted; proposal.viewed; proposal.revised; proposal.withdrawn; proposal.expired	Sales, Procurement, notifications, analytics
Review	proposal.clarification_requested; proposal.revision_requested; proposal.shortlisted; proposal.preferred; proposal.negotiation_started; proposal.declined	Parties, evaluation, notifications
Decision	proposal.awarded; proposal.contract_drafted; proposal.contracted	Award orchestrator, Contract, ontology, notifications
Commercial	vendor.linked_or_created; customer.linked_or_created; opportunity.linked_or_created; contract.draft_created; contract.activated; transaction.recorded	Catalogue, Sales, Procurement, intelligence, reports

7.3 Canonical Proposal Status Transitions

From	Permitted next state	Actor / condition
DRAFT	SUBMITTED	Proposer with submit permission; validation and paid entitlement pass
SUBMITTED	VIEWED or WITHDRAWN or EXPIRED	Recipient view; proposer withdrawal; system expiry
VIEWED	UNDER_REVIEW, CLARIFICATION_REQUESTED, REVISION_REQUESTED, DECLINED	Recipient reviewer
UNDER_REVIEW	CLARIFICATION_REQUESTED, REVISION_REQUESTED, SHORTLISTED, DECLINED	Recipient reviewer
CLARIFICATION_REQUESTED	UNDER_REVIEW, REVISION_REQUESTED, WITHDRAWN	Response/recipient action/proposer withdrawal
REVISION_REQUESTED	REVISED, WITHDRAWN, EXPIRED	Proposer submits new version or terminates
REVISED	VIEWED, UNDER_REVIEW, SHORTLISTED, DECLINED	Recipient reviewer; exact new version
SHORTLISTED	PREFERRED, UNDER_REVIEW, DECLINED	Recipient reviewer/decision authority
PREFERRED	NEGOTIATION, SHORTLISTED, DECLINED	Recipient decision authority
NEGOTIATION	REVISED, AWARDED, DECLINED, WITHDRAWN	Authorised parties; Award requires award permission
AWARDED	CONTRACT_DRAFTED	Award orchestrator only
CONTRACT_DRAFTED	CONTRACTED, NEGOTIATION, ARCHIVED	Existing Contract workflow or authorised recovery
CONTRACTED	ARCHIVED	Authorised archival only
DECLINED/WITHDRAWN/EXPIRED	ARCHIVED	Authorised party/system policy

From	Permitted next state	Actor / condition
ARCHIVED	None	Read-only terminal history

7.4 Event Processing Rules

27. Events contain identifiers and safe metadata, never proposal bodies, private financial data, credentials, access tokens or raw uploaded content.
28. Consumers are idempotent and tolerate at-least-once delivery. Ordering-sensitive consumers use aggregate version/sequence.
29. Side effects such as notifications, Saved Documents indexing and commercial orchestration occur from committed events/outbox records, not uncommitted UI state.
30. Schema changes create a new eventVersion with backward-compatible consumer migration; do not silently change version 1 semantics.
31. Cross-tenant consumers receive only the minimum safe projection appropriate to the relationship.

8. Screen-by-Screen Acceptance Criteria

Screen	Location	Required content/actions	Acceptance criteria
Proposal settings	Business/Marketplace settings	Mode multi-select; categories/exclusions; eligibility/caps; derived status preview; Save	Authorised save persists/audits; invalid combinations explain errors; none selected derives Not Currently Open; loading/error/permission states; keyboard labels.
Procurement requests list	Business Operations - Procurement	Draft/Published/Paused/Closed filters; counts; Create Request	Tenant-scoped pagination; empty/no-results/error states; role-safe actions; canonical status only.
Request builder	Business Operations - Procurement	General/Specific fields; requirements; criteria; deadline; visibility; cap; AI drafting option	Type-specific validation; draft autosave/explicit save; Publish confirmation; stale-edit conflict; accessible form errors.

Screen	Location	Required content/actions	Acceptance criteria
Marketplace card/profile	Marketplace	Open for Proposals badge; accepted modes/categories; Apply	Derived status matches preference; public projection excludes private data; Apply unavailable when closed/ineligible.
Request detail	Marketplace	Recipient, need/scope, deliverables, eligibility, evidence, deadline, budget visibility, Apply	Published revision shown; invitation/deadline/cap checks; safe expired/closed state; deep link works.
Method chooser	Marketplace Apply	Generate vs Upload; value statements; paid disclosure	Focus trapped/restored; route context preserved; disclosure visible; no action taken on modal open.
Contextual signup	Generate or Upload	Minimal account creation; saved opportunity/progress notice	Generate requires signup before upgrade; Upload requires signup at Submit; failures preserve safe draft state.
Contextual upgrade	Proposal-specific payment	Receiving business/request; Starter Insight preselected; monthly/annual; Compare Plans	No generic pricing redirect; return state signed/validated; payment cancel/fail preserves progress; paid user bypass.
Blueprint proposal generator	Business Blueprints - Proposals	Offering selection; mapped data; missing questions; generation progress; editor; preview; approve	Free users blocked; approved sources/provenance visible; retry safe; user approval mandatory before submit.
Upload and extraction	Apply flow	Drop zone; restrictions; scan/extraction progress; field confidence; corrections; preview	Unsafe file blocked; anonymous token isolated; prompt injection ignored; original preserved; expiry explained.
Submission review	Before Submit	Recipient/request; exact version; commercial summary; disclosures; consent; Submit	All validation results; one-active rule; idempotent double-click; stale/closed/cap/plan errors preserve draft.

Screen	Location	Required content/actions	Acceptance criteria
Recipient proposal inbox	Business Operations - Procurement	Unread/status/deadline/Offering filters; summary; shortlist	Recipient tenant only; stable pagination; no proposer-private analysis; accessible table/list alternative.
Proposer proposal activity	Business Operations - Sales	Recipient/request/status/updated; actions permitted	Same canonical status as Procurement; revisions not duplicate proposals; tenant/role isolation.
Proposal detail and thread	Sales/Procurement deep link	Structured data; source; versions; timeline; clarification/revision; status actions	Role-filtered fields/actions; cleared downloads only; exact version; all material actions audited.
Comparison	Procurement shortlist	Mandatory results; criteria scores; price/terms; missing data; confidence; trade-offs	Only authorised shortlisted versions; explicit units/currency; no automatic decision; insufficient data clearly labelled.
Scenario results	Procurement intelligence	Base/downside/expected/upside; engine deltas; assumptions; risks; confidence	Hypothetical styling; no live mutation; exact version/ruleset/model shown; missing engine outputs not fabricated.
Award confirmation	Procurement	Exact proposal version; rationale; downstream automation notice; confirmation	Award permission + recent auth if existing policy; AI cannot invoke; stale/double action safe; audit reason retained.
Contract handoff	Business Operations - Contracts	Draft fields; linked Vendor/Customer/Offering/proposal; orchestration state	Draft only; human review/activation; partial failure/retry visible; no duplicate records.
Saved Documents	Business Blueprints	Type/status/owner/date filters; Open/Continue/Preview/Export/Archive; operational usage links	Operational status read-only; archive does not alter active workflow; tenant and role isolation.
Reports	Business Operations - Reports	Pipeline, response, shortlist, award, contract and quality measures	Canonical reconciliation; privacy-safe aggregates; access/export permissions; time/currency filters explicit.

9. Security, Privacy and Tenancy Requirements

9.1 Authentication and Authorisation

32. Use the existing authentication provider and session controls. Proposal Intelligence must not introduce an independent identity store.
33. Resolve current user and tenant server-side. Never trust business_id, tenant_id, role, plan or ownership values from the browser.
34. Authorise every object and action, including list filters, attachments, versions, messages, evaluations, downloads, awards and retries.
35. Cross-tenant proposal access is relationship-based: the user must belong to the proposer or recipient tenant and hold the action permission. A resource ID alone grants nothing.
36. Moderator/support access uses a separate explicit policy, reason, audit event and least-privilege projection. Ordinary administrators do not automatically receive proposal bodies.
37. Sensitive award/contract activation follows existing reauthentication or approval rules if present; do not weaken them.

9.2 Tenant Isolation

Control	Requirement
Query scope	All repository/service queries apply tenant/party scope before filtering by resource ID. Tests attempt valid-ID substitution across tenants.
Composite ownership	Prefer composite tenant + entity constraints or established repository policy so cross-tenant foreign-key swaps cannot persist.
Cache	Cache keys include tenant and role-safe projection; public projections are separate from authenticated/private results.
Jobs/queues	Job payload carries authoritative entity IDs and re-resolves tenant/permissions server-side; do not serialize browser claims as authority.
Search/index	Indexes carry tenant/visibility filters; public Marketplace index contains only approved public projection.
Object storage	Keys are opaque; access uses short-lived authorised URLs or streaming endpoint; storage bucket/path is not public.

Control	Requirement
Analytics/logs	No proposal body, private score, financial input or raw prompt is included; tenant identifiers follow existing pseudonymisation policy.
Backups/export/delete	Follow platform privacy/retention controls while preserving legally/audit-required submitted versions and decisions.

9.3 File and AI Security

38. Allow only server-detected PDF and non-macro DOCX; enforce approved size before and during upload; reject archives, executables, HTML and active content.
39. Quarantine first, malware scan, then extract in a sandbox without network or code execution. Do not follow embedded URLs or load remote resources.
40. Treat document text, metadata and embedded instructions as untrusted data. System/developer policies are separated from extracted content; tool use is not driven by document instructions.
41. Sanitise preview rendering and message content against stored/reflected XSS; use safe download disposition and MIME headers.
42. Use checksums for duplicate detection and integrity. Near-duplicate/relevance scoring must not expose other tenants' proposal content.
43. Expire and securely delete abandoned anonymous uploads under the approved retention period. Never retain them indefinitely for marketing.
44. Store model, prompt-template and ruleset versions plus safe provenance. Do not store secrets, raw credentials or hidden system prompts in user-visible records.
45. AI output cannot perform a commercial action directly. Server-side action APIs require a human user session, permission and explicit confirmation.

9.4 Abuse, Rate and Commercial Controls

46. Rate-limit public discovery, upload-session creation, file upload, extraction polling, signup claim, generation, messages, invitations and report actions by IP/session/account/tenant as appropriate.
47. Enforce one active proposal per business per request, excluding revisions, at database and service layers.
48. Apply plan allowances across different requests and trust levels; revisions do not consume another proposal allowance.

49. Support recipient caps, deadlines, invite-only requests, block/report, duplicate detection, suspension and appeal.
50. Payment webhook/entitlement is authoritative; browser success pages cannot grant access. Handle webhook replay and delayed settlement idempotently.

10. Workflow Test Catalogue

Test IDs are stable references for tickets and release evidence. Automate at the lowest suitable level and retain a smaller set of browser-level end-to-end tests for critical journeys.

10.1 Settings, Requests and Marketplace

Test ID	Scenario	Expected result
T-SET-01	Authorised business enables Unsolicited + Solicited General	Settings persist, audit version created and status derives Open to All Relevant Proposals.
T-SET-02	No proposal mode selected	Profile derives Not Currently Open and Apply is unavailable.
T-SET-03	Unauthorised member updates preference	403; no change/event.
T-REQ-01	Create and publish Specific request	Required fields/criteria freeze in revision; public projection appears.
T-REQ-02	Publish General request without required outcome/category	422 field errors; remains Draft.
T-REQ-03	Revise published request	New immutable revision; active proposers notified; old submissions retain answered revision.
T-REQ-04	Apply to closed/expired/cap-reached request	Safe conflict; no upload/submission created.
T-MKT-01	Public reads profile/request	Only public fields; no tenant IDs/private criteria/preferences.
T-MKT-02	Invite-only request opened without invitation	Safe not-found/denied response without existence leakage.

10.2 Upload, Signup and Subscription

Test ID	Scenario	Expected result
T-UPL-01	Anonymous user uploads clean PDF	Quarantined, scanned, extracted and previewed under opaque session.
T-UPL-02	Upload macro DOCX/executable/oversize	Rejected before extraction; safe error; audit/metric recorded.
T-UPL-03	Malware scanner blocks file	File remains inaccessible; extraction and download never run.
T-UPL-04	Document contains prompt injection	Text is extracted as data; system behaviour/policies unchanged.
T-UPL-05	User accesses another anonymous token	No disclosure; rate/security event recorded.
T-SGN-01	Anonymous upload claimed after signup	Single atomic claim; exact progress restored.
T-SGN-02	Second account reuses claimed token	Claim rejected; original ownership unchanged.
T-SUB-01	Free user clicks Submit	Contextual upgrade shown; proposal retained; no delivery.
T-SUB-02	Payment success/webhook replay	Entitlement granted once; return to final confirmation; no duplicate charge/outcome.
T-SUB-03	Payment cancel/failure	Draft preserved; retry offered; no submission.
T-SUB-04	Existing paid user uploads and submits	No upgrade screen; all validations pass; one frozen version delivered.

10.3 Submission, Status and Communication

Test ID	Scenario	Expected result
T-PSL-01	Two concurrent submits with same idempotency key	Same canonical submission/version response.
T-PSL-02	Concurrent different keys for same proposer/request	Database allows only one active submission; safe conflict returns canonical link.

Test ID	Scenario	Expected result
T-PSL-03	Revision after recipient request	New version number under same submission; earlier checksum/payload unchanged.
T-PSL-04	Revision submitted without request/permission	Rejected; no version/status event.
T-STS-01	Client attempts arbitrary status write	Endpoint unavailable/rejected; transition service is sole writer.
T-STS-02	Valid Submitted -> Viewed -> Under Review	Actor/prior/next/version/time events recorded.
T-STS-03	Terminal proposal shortlisted	Rejected as invalid transition.
T-MSG-01	Clarification/response between parties	Typed messages visible to authorised parties; notifications deduplicated.
T-MSG-02	Proposer tries to read recipient-only evaluation	Safe 404/403; no cache/log leakage.
T-NTF-01	Event is delivered twice	One notification outcome; consumer records idempotent processing.

10.4 Generation and Saved Documents

Test ID	Scenario	Expected result
T-GEN-01	Non-user chooses Generate	Signup first, opportunity saved, contextual upgrade shown; no generation on Free.
T-GEN-02	Paid user selects another tenant's Offering ID	Denied; no source data returned to model/UI.
T-GEN-03	Missing required capability evidence	Question/confirmation required; generator does not invent evidence.
T-GEN-04	Generation job retry/timeout	One logical job/cost; draft recoverable; no submission side effect.
T-GEN-05	User edits and approves generated proposal	Approved candidate records provenance and is ready for validation.

Test ID	Scenario	Expected result
T-DOC-01	Proposal draft saved	DocumentRecord created/updated and linked to authoring document.
T-DOC-02	Saved Document archived while submission active	Document index archived only; operational proposal remains intact.

10.5 Evaluation and Intelligence

Test ID	Scenario	Expected result
T-EVL-01	Mandatory requirement fails	Failure remains visible; weighted score cannot convert it to Pass.
T-EVL-02	Evidence is ambiguous	Result is Not Determined with reduced confidence, not Pass.
T-EVL-03	Same proposal/ruleset evaluated twice	Deterministic scores match; narrative variation cannot change scores.
T-CMP-01	Compare shortlisted proposals with different currencies	Explicit conversion policy/rate/time or comparison blocked; no silent normalisation.
T-CMP-02	AI recommends low-confidence proposal	Low confidence/missing data prominent; user action still required.
T-SCN-01	Run four scenarios	Each references exact version/assumptions; results labelled hypothetical.
T-SCN-02	Scenario/evaluation state snapshot	No live invoice, expense, cash, KPI, Contract or Fragility source record changes.
T-SCN-03	Unsupported engine metric	Displays Not Available; does not fabricate a delta.
T-REC-01	Recommendation service response	Reasons, trade-offs, assumptions, missing data, confidence and risks are returned with provenance.
T-REC-02	Model attempts action call	No shortlist/prefer/award capability exists in model execution context.

10.6 Award and Commercial Conversion

Test ID	Scenario	Expected result
T-AWD-01	Reviewer without award permission attempts Award	403; no decision/event/orchestration.
T-AWD-02	Authorised user awards stale version	409; current exact version shown for reconfirmation.
T-AWD-03	Double-click/retry Award	One ProposalDecision and one orchestration run.
T-VEN-01	Vendor already exists by Business ID	Existing Vendor linked; no duplicate/name matching.
T-VEN-02	Vendor absent	Created once from approved Marketplace data within requester Catalogue.
T-CUS-01	Customer link/create in proposer Catalogue	Correct reciprocal record, tenant scope and audit.
T-CON-01	Award orchestration completes	One Contract draft with correct exact-version fields/links; status Contract Drafted.
T-CON-02	Contract creation fails after Vendor/Customer success	Award remains; run shows partial failure; safe retry reuses existing records.
T-CON-03	System/AI attempts Contract activation	Rejected; authorised human approval required.
T-CON-04	Existing Contract is activated	Proposal becomes Contracted; confirmed commitment event emitted once.
T-TRX-01	Award without Contract activation	No actual financial/Fragility mutation.
T-TRX-02	Actual transaction recorded	Existing transaction validation applies and actual intelligence sources update through established event.

10.7 Tenancy, Security and Non-functional

Test ID	Scenario	Expected result
T-TEN-01	Swap request/submission/version IDs across two tenants	Every list/detail/action/download returns safe denial; no timing/body leakage beyond policy.
T-TEN-02	Cross-tenant cache attempt	Cache key/projection prevents serving another tenant's result.

Test ID	Scenario	Expected result
T-TEN-03	Background job payload is tampered	Job re-resolves authoritative ownership and rejects mismatch.
T-SEC-01	Stored/reflected XSS in filename, message and extracted field	Rendered safely; no script execution.
T-SEC-02	Expired signed download/session token	Access denied; no fallback public URL.
T-SEC-03	Rate limit burst on upload/generation/messages	Policy response with retry guidance; other tenants unaffected.
T-SEC-04	Logs/traces sampled	No proposal body, raw prompt, private financial data or credentials.
T-ACC-01	Keyboard-only critical workflow	All controls reachable/operable; focus visible and restored.
T-PERF-01	Inbox/Marketplace expected dataset	Meets repository budget; pagination/indexes avoid unbounded scans.
T-REL-01	Queue/service restart during generation/orchestration	Jobs resume/retry idempotently; no duplicate records or lost frozen versions.

11. Definition of Done

11.1 Every Ticket

51. Scope and acceptance criteria are implemented without unrelated refactoring or later-phase work.
52. Authorisation and tenant isolation are enforced server-side and tested for positive and negative paths.
53. Input schemas, domain errors, concurrency and idempotency rules are implemented where relevant.
54. Unit/integration tests and the identified workflow/security tests pass; failures and Not Run checks are disclosed.
55. Formatting, lint, type-check, production build, migration validation and git diff --check pass under repository-standard commands.

56. Database migration is additive/forward-safe, reviewed and paired with deployment/disable/rollback notes when applicable.
57. Audit event, logs, metrics and traces are privacy-safe and use correlation IDs.
58. Loading, empty, error, permission, stale/conflict and retry states are implemented for UI work.
59. Accessibility is verified for keyboard, labels, focus, errors, status announcements and contrast where applicable.
60. Documentation, API schemas and traceability matrix are updated; final report lists changed files and acceptance evidence.

11.2 Phase Gate Evidence

Gate	Evidence required
Architecture	Technical baseline, ADRs, dependency review, migration plan, threat model and unresolved decisions.
Functional	Ticket acceptance matrix, critical workflow recordings/screenshots where useful and product UAT sign-off.
Quality	Test reports, coverage trend, production build, migration checks and known limitations.
Security/privacy	Tenant isolation suite, upload/AI controls, permission tests, log review and security owner approval.
Reliability	Idempotency/concurrency results, queue/job recovery, partial-failure recovery and observability evidence.
Release	Feature-flag/cohort plan, data migration/runbook, rollback/disable procedure, support readiness and go/no-go record.

12. Product Decisions to Confirm During Phase 0

Decision	Recommended MVP default	Why it must be explicit
Plan allowances	Define requests-per-billing-period by paid tier; revisions excluded.	Required for entitlement, abuse and analytics tests.
Anonymous upload retention	Short disclosed TTL with automatic deletion if unclaimed.	Controls privacy, cost and session recovery.

Decision	Recommended MVP default	Why it must be explicit
Maximum file size	Choose from infrastructure/security benchmark and display before upload.	Affects transfer, scanning and extraction budgets.
Award multiplicity	Single award per request in MVP unless Product explicitly enables multi-award.	Determines uniqueness, statuses and Contract orchestration.
Budget visibility	Recipient selects Hidden, Range or Exact.	Controls Marketplace projection and proposal comparison.
Published request revision	Notify active proposers, retain answered revision and allow deadline adjustment/withdrawal.	Required for fairness and immutable evaluation context.
Evaluation weights	Versioned defaults with authorised recipient adjustment before evaluation.	Required for reproducible comparison.
Moderation SLA	Pilot-specific owner and response target before public launch.	Required for trust and incident handling.
Contract signature	Use existing Contract workflow only; external e-signature is a separate project.	Prevents scope expansion and unintended binding action.

13. Recommended First Instruction to Claude Code

```
You are working on EnterprateAI Proposal Intelligence.

Read, but do not yet implement, the following controlling documents:
- Proposal Intelligence PRD v1.1
- Proposal Intelligence Claude Code Implementation Pack v1.0
- Adopted UI/wireframes
- The repository CLAUDE.md and all scoped engineering instructions

Perform ticket PI-000 only: Repository architecture discovery.

Inspect the real repository and produce docs/proposal-intelligence/technical-baseline.md
covering:
1. Framework, runtime, package manager and deployment.
2. Current module/navigation architecture.
3. Database, ORM, schema, migrations and transaction patterns.
4. Authentication, tenant resolution, roles and permissions.
5. Catalogue Offering/Customer/Vendor and existing Contract models/services.
6. Marketplace, Business Blueprints, Business Operations, payments/entitlements and Saved
Documents patterns.
7. Object storage, uploads, malware scanning, AI jobs, queues and event/outbox patterns.
8. Existing test, lint, type-check, build and security commands.
9. Exact reusable components and any genuine gaps/conflicts.
10. Proposed file/module placement for Phase 1, without creating feature code.
```

```
Do not install packages, change schema, edit production code or begin PI-001. Cite repository
paths for every conclusion. Finish with open questions and recommended ADRs.
```

14. Handoff Checklist

61. Share PRD v1.1, this pack and the adopted UI screens with the team.
62. Place the Markdown pack in the repository documentation folder.
63. Confirm the product decisions in Section 12 and name decision owners.
64. Run PI-000 and review Claude Code's architecture baseline before any feature code.
65. Create the Phase 0 tickets in the team's tracker and assign engineering, product, security and data reviewers.
66. Approve Phase 0 before starting Phase 1; implement one ticket at a time.
67. Retain the feature flags and release gates throughout development.